NASCON '26 — App Dev Competition

Part II: Live Streaming, Dynamic QR Deep-Linking & Automated Grid QA

Problem Specification — Day 2 Final Sprint

Active Window	Sunday, Oct 18 (10:15 AM – 02:00 PM PKT)	**Evaluation**	Automated Suite + Finalist Viva
Venues	**C-401 – C-404**	**Prizes**	Winner: PKR 35,000 | Runner-Up: PKR 15,000
Code Freeze	14:00:00 PKT Sharp	**Final Push**	14:05:00 PKT Sharp (Hard Lock)
Weightage Split	**80% Backend Platform & Automated QA Infrastructure | 20% Frontend (Mobile + Guest Web UI)**		

Binding Contract Notice: This document contains the complete, exclusive, and legally binding API and QA testing contract for Day 2. Automatic GitHub Actions autograders, container probes, Selenium Grid 4, Appium 2, k6 runners, and the Top-8 Finalist Viva evaluate against these exact schemas, DOM selectors, status codes, and network parameters. If any verbal statement conflicts with this document, this document wins.

Contents

1 Project Overview & Architectural Continuity **2**

1.1 Deliverable Surfaces . . . 2
1.2 Phase II Commit Continuity and Cryptographic Salt . . . 2
1.3 Repository Structure, GitHub Actions & Push Checkpoints . . . 3

2 Phase II Engineering Outcomes **3**

3 Functional Requirements (Part II) **4**

3.1 Creator Tools & Live Streaming Engine . . . 4
3.2 Zero-Friction Guest Access & Progressive Onboarding . . . 4
3.3 Real-Time Notifications, Moderation & Observability . . . 5

4 Non-Functional Requirements (Part II) **6**

5 Mandatory Autograder Interface (`/audit/v1/*`) **6**

5.1 Phase II Endpoint Index . . . 7
5.2 Global Conventions, Startup & Reset Extensions . . . 7
5.3 Phase II Signature Specification (DAY2_SALT) — Exact Recipe . . . 8
5.4 Stream Key Lifecycle Management . . . 8
5.5 Synchronized 4-Way Co-Stream Orchestration . . . 9
5.6 Dynamic Cryptographic QR Deep-Linking & Exports . . . 10
5.7 Guest Read-Only Boundary & Progressive Cart Merge . . . 11
5.8 Chat Moderation Engine & Notification DLQ Probe . . . 12
5.9 Observability (`/metrics`) & Privacy Erasure Probes . . . 13

6 Automated Grid QA Harness (Selenium Grid 4, Appium 2, k6 & CI) 13
6.1 Mandatory Standard Port Conventions . . . 13
6.2 Selenium Grid 4 Architecture & Mandatory DOM Selectors . . . 14
6.2.1 Docker Compose Rules for Selenium Grid 4 . . . 14
6.2.2 Mandatory Guest Web Player DOM Contract (`data-testid`) . . . 15
6.2.3 Required Selenium Test Cases (`tests/selenium/test_guest_grid.py`) . . . 15
6.3 Appium 2 Mobile Automation & Deep-Link Standards . . . 16
6.4 k6 High-Concurrency Load Specification (NFR-2) . . . 16
6.5 Multi-Laptop Workload Distribution Architecture (Labs C-401 – C-404) . . . 17

7 Phase II Evaluation Weightage (80% Backend / 20% Frontend) 17
7.1 Mandatory 80% Backend / 20% Frontend Marks Distribution . . . 17
7.2 Automated Vector Evaluation & Coverage File (`COVERAGE_DAY2.json`) . . . 18
7.3 Disqualification & Compliance Rules . . . 18

8 Phase II Hidden Complexity Scenarios 18

A Reference Code and Cryptographic Test Vectors 19

B Error Code Catalogue (Part II) 20

C Pre-Submission Final Checklist (Labs C-401 – C-404) 20

1. Project Overview & Architectural Continuity

Phase II extends your locked Phase I multi-sport league and club-operations platform with low-latency creator broadcasting tools, synchronized 4-participant co-streaming, zero-friction mobile web guest access via cryptographic dynamic QR deep-links, progressive guest-to-member cart merge, automated chat moderation with Dead-Letter Queue (DLQ) observability, and an automated testing harness powered by **Selenium Grid 4**, **Appium 2**, **grafana/k6**, and **GitHub Actions**.

1.1 Deliverable Surfaces

Phase II evaluates three integrated deliverable surfaces built inside your existing repository:

1. **Backend Platform (80% Weightage Pool):** Your existing REST API on port `8080` extended with Phase II `/audit/v1/*` endpoints, a self-hosted media server (MediaMTX / LiveKit / SRS), a real-time WebSocket/SSE message bus, Prometheus `/metrics` exporter, and an automated chat moderation & notification DLQ engine.
2. **Native Mobile Application (Part of 20% Frontend Pool):** Your Phase I Android/iOS client updated with solo stream broadcasting, 4-way co-stream split-view, native QR code scanner & Android Intent-Filter (`nascon://...`) handling, and real-time push notifications.
3. **Guest Web Player (New — Part of 20% Frontend Pool):** A mobile-optimized, zero-install, zero-account responsive web application served on `http://localhost:8080/web/*` and opened via dynamic QR scans. It must render HLS/WebRTC video, display real-time match scores, showcase pinned merchandise, enforce server-side read-only boundaries for anonymous guests, and support progressive registration with cart migration.

1.2 Phase II Commit Continuity and Cryptographic Salt

- **Commit Continuity (`DAY1_BASE_SHA`):** Your Phase II commits must build directly on top of your locked Saturday 15:05 PKT commit hash. The GitHub Actions autograder clones your repository and runs a git

ancestry check (`git merge-base --is-ancestor $DAY1_BASE_SHA HEAD`) and executes a regression pass on Phase I unit tests and baseline `/audit/v1/reset` probes. Re-initializing the repository or wiping Phase I history incurs an automatic penalty.

- **DAY2_SALT Enforcement:** Every generated stream key record, co-stream session state, dynamic QR link payload, guest cart-merge receipt, and moderation audit record must include a deterministic HMAC-SHA256 signature computed using your team's unique 16-character DAY2_SALT printed on your Day 2 venue slip:

$$\texttt{signature} = \text{HMAC-SHA256}\left(\texttt{DAY2_SALT},\ \texttt{entity_id} + \texttt{"|"} + \text{canonical}(\texttt{payload})\right)$$

  The exact byte-level recipe for `entity_id` and `canonical(payload)` for each endpoint is defined in Section 5.3 and Appendix A. Follow it literally.

1.3 Repository Structure, GitHub Actions & Push Checkpoints

Your public GitHub repository registered at your venue desk (**C-401 – C-404**) must contain the following additional Phase II paths at the repository root:

Path (repo root)	Requirement
`docker-compose.yml`	Brings up the full Phase I + Phase II backend stack (Core API on `8080`, DB, Redis, MinIO, Media Server on 1935/8888/8889, and Selenium Grid 4 Hub + 3 Browser Nodes on 4444). Must forward both `DAY1_SALT: ${DAY1_SALT}` and `DAY2_SALT: ${DAY2_SALT}`.
`.github/workflows/autograde.yml`	GitHub Actions workflow triggering on every push to `main`, executing backend contract probes, Selenium Grid 4 node verification, and k6 smoke checks.
`tests/selenium/`	Parallel Selenium Grid 4 WebDriver test suite verifying Guest Web Player flows (Section 6.2).
`tests/appium/`	Appium 2 mobile automation test suite verifying native deep-linking and cross-surface streaming (Section 6.3).
`tests/k6_load.js`	Runnable k6 concurrency load script verifying 500 concurrent guest viewers and 50 chat posters (Section 6.4).
`tests/reports/`	Output directory containing generated JUnit XML / Allure / HTML test execution reports (`selenium-report.xml`, `appium-report.xml`, `k6-summary.json`).
`COVERAGE_DAY2.json`	Line-coverage percentages for the five Phase II backend modules (Section 7.2).

GitHub Push Checkpoints: At least one commit must be pushed (verified by GitHub server push timestamp) inside each of these four windows on Sunday, Oct 18:

- **Window 1:** 10:15 AM – 11:30 AM PKT
- **Window 2:** 11:30 AM – 12:45 PM PKT
- **Window 3:** 12:45 PM – 01:45 PM PKT
- **Window 4 (Final Code Freeze):** 01:45 PM – 02:00 PM PKT (Grace push deadline: **14:05:00 PKT Sharp**)

Each missed checkpoint window incurs an automated checkpoint penalty. The autograder checks out the last commit pushed at or before **14:05:00 PKT**; any commit pushed after 14:05:00 PKT is completely ignored.

2. Phase II Engineering Outcomes

By completing Part II, teams demonstrate the ability to:

- Integrate low-latency real-time video ingest (RTMP/WebRTC WHIP) and playback (HLS/WebRTC WHEP) with cryptographic stream key lifecycle management;
- Orchestrate multi-participant 4-way co-streaming sessions with deterministic host-disconnect handoff and unified chat sequencing;
- Implement dynamic cryptographic QR deep-linking that adapts target routes as match states evolve

without re-printing physical QR assets;
- Enforce strict server-side zero-trust boundaries between anonymous guest tokens and authenticated member sessions while supporting seamless cart migration;
- Build resilient real-time notification pipelines backed by Dead-Letter Queues (DLQ) and automated leet-speak/flood chat moderation;
- Execute automated multi-node browser, mobile, and load verification using Selenium Grid 4, Appium 2, and k6 under strict hardware resource constraints.

3. Functional Requirements (Part II)

Maintain `TRACEABILITY.md` at the repository root mapping every Phase II requirement ID below to its corresponding source files and automated tests.

3.1 Creator Tools & Live Streaming Engine

FR-3.1 Public Athlete Identity & Concurrency-Safe Social Graph

- Athlete profile view aggregating career match records (from Phase I event logs), active club affiliations, upcoming fixture schedule, and an indexed VOD archive of completed live streams.
- Concurrency-safe follow/unfollow operations guaranteeing zero follower-count drift under simultaneous toggle bursts, with idempotent state resolution per `(follower_id, athlete_id)` pair.

FR-3.2 Low-Latency Solo Streaming & Dynamic Overlays

- In-app live broadcast via WebRTC (WHIP) or RTMP to your self-hosted media server (glass-to-glass latency target < 3.0 seconds on local LAN) with HLS (`.m3u8`) and WebRTC playback.
- External ingest compatibility (OBS Studio / FFmpeg) authenticated via rotating stream keys signed with `DAY2_SALT`. Revoking or rotating a stream key must immediately sever active RTMP/WebRTC ingest pipelines within 2.0 seconds.
- Real-time dynamic overlays: Live match score (from FR-1.1), club badges, and pinned merchandise items (from FR-2.3) rendered dynamically over the video player via WebSocket/SSE metadata events without restarting the media stream.
- Automated stream recording to self-hosted object storage (MinIO) and immediate VOD indexing upon stream termination.

FR-3.3 Synchronized 4-Way Co-Streaming & Host Handoff

- Multi-broadcaster split-screen (`grid_2`, `grid_4`, or `spotlight` layout supporting up to 4 concurrent participants) mixed via SFU (e.g., LiveKit) or server-side FFmpeg/MediaMTX composition outputting a unified composite playback URL.
- Host moderation controls: Generate cryptographically signed invite tokens for squad members; mute audio, spotlight video, or kick participants in real time.
- Unified Audience Chat: Exactly one synchronized chat room (`chat-{session_id}`) per co-stream with deterministic monotonically increasing sequence numbers across all hosts and viewers.
- Deterministic Host Disconnect Handoff: If the primary host disconnects abruptly, hosting privileges must automatically migrate to the next participant in the `participants` array order within 1.5 seconds, or terminate cleanly if zero participants remain, leaving zero orphaned FFmpeg/WebRTC zombie processes.

3.2 Zero-Friction Guest Access & Progressive Onboarding

FR-4.1 Dynamic Cryptographic QR Deep-Linking

- System-generated dynamic QR codes for any seeded or newly created match (`match-*`), club (`club-*`), player (`player-*`), or co-stream session (`cs-session-*`).
- Each QR code encodes a stable short URL (`http://localhost:8080/q/{short_code}`) signed with `DAY2_SALT` that resolves server-side via HTTP `302` Found (or JSON resolution on the audit probe) based

on live entity state:
  - Match `upcoming` → `/web/matches/{id}/preview`
  - Match `in_progress` → `/web/matches/{id}/live`
  - Match `completed` → `/web/matches/{id}/vod`
  - Match `postponed` / `cancelled` / abandoned → `/web/matches/{id}/notice?state={status}`
- Vector & Raster Export: Every QR code must be exportable on demand as valid SVG (`image/svg+xml`), PNG (`image/png`), and print-ready PDF (`application/pdf`).
- Privacy-Safe Scan Telemetry (NFR-9): Tracks total scan count, device family (`mobile` or `desktop` derived from User-Agent), and UTC timestamp bucket without storing client IP addresses, cookies, or PII.
- Hybrid Deep-Link Routing: When scanned on a mobile device with the native app installed, Android App Links / Custom Scheme Intent-Filters (`nascon://entity/{entity_id}`) launch the native activity directly; otherwise, the browser opens the Guest Web Player.

FR-4.2 Zero-Account Guest Experience

- Opening a resolved QR route in any mobile or desktop browser launches the Guest Web Player immediately with zero app installation prompts, zero forced login walls, and zero credential dialogs.
- The backend issues a short-lived anonymous viewer token (`guest_anon_tok_*`) scoped strictly to read-only stream playback, live score subscription, and local guest cart staging.

FR-4.3 Server-Enforced Read-Only Boundary

- Any attempt by a request bearing a `guest_anon_tok_*` token (or unauthenticated guest session) to invoke a state-mutating write endpoint (posting chat messages, following an athlete, submitting a direct checkout order, or mutating rosters/scores) must be rejected server-side with HTTP `403 Forbidden` and machine-readable error code `GUEST_WRITE_FORBIDDEN`. Client-side UI hiding alone receives zero credit.

FR-4.4 Progressive Registration & Atomic Cart Merge

- Guests may browse pinned merchandise and stage items in a temporary guest cart associated with their `guest_token`.
- Prompting for registration or login occurs only when the guest explicitly initiates a protected write action (e.g., clicking "Proceed to Checkout" or "Send Chat").
- Upon authenticating as a registered user, the backend atomically merges all staged `guest_cart` items into the registered user's persistent cart (summing quantities if the same `variant_id` already exists, capped by available variant stock), invalidates the `guest_token` against repeat merges (idempotent replay returns the merged cart without double-adding), and redirects the client back to the exact stream playback timestamp.

3.3 Real-Time Notifications, Moderation & Observability

FR-5.2 Notification Bus & Dead-Letter Queue (DLQ)

- Real-time event dispatch via WebSocket/SSE for live score updates, co-stream invites, and order status transitions.
- Any notification delivery that fails (e.g., simulated downstream webhook/push failure or disconnected target subscriber with `simulate_dlq: true`) must undergo bounded exponential retry (max 3 attempts) before routing deterministically to a persistent Dead-Letter Queue (DLQ) inspectable via the audit contract.

FR-5.3 Automated Chat Moderation Engine

- Synchronous pre-broadcast filtering on all incoming chat messages using normalized regex and token matching:
  - **Leetspeak & Obfuscation Normalization:** Convert `@`→a, `0`→o, `1/!`→i, `3`→e, `$`→s, strip zero-width/punctuation separators (e.g., `b@d_w0rd`, `p.0.r.n`, `s!c@m`, `f*u*c*k`, `a$$h0le`), and replace matched profanity tokens in the message text with `***` while tagging `"flagged": true, "reason": "profanity_leetspeak"`.

- **Character Flood & Rate Limiting:** Any message containing ≥ 4 consecutive identical non-alphanumeric characters or ≥ 5 repeated characters (e.g., `"GOAL!  AAAA!!!"`) or exceeding 3 messages within a 2-second sliding window from the same sender is blocked from broadcast, tagged `"flagged": true, "reason": "flood_character_repeat"` (or `"rate_limit_exceeded"`), and the offending sender is appended to `kicked_users` (and `muted_users`).

4. Non-Functional Requirements (Part II)

ID	Requirement	Minimum Acceptance Bar & Verification Method
NFR-1	**Real-Time Score Latency**	Score events appended via POST `/audit/v1/events` must broadcast over WebSocket (`ws://localhost:8080/ws/matches/{match_id}`) or SSE (GET `/api/v1/matches/{match_id}/stream`) and reach 95% of connected clients within < 500 ms ($p_{95} < 500$ ms). Verified via a 25-client concurrent WebSocket probe.
NFR-2	**High-Concurrency Scale**	Backend and media proxy must sustain **500 concurrent guest viewers** polling/streaming QR & HLS manifests plus **50 concurrent authenticated chat posters** for 15 seconds with **0% HTTP 5xx errors** and p_{95} HTTP response latency < 800 ms. Verified via k6 run `tests/k6_load.js`.
NFR-6	**System Observability**	Expose GET `/metrics` in standard Prometheus text exposition format (`text/plain; version=0.0.4`) reporting at minimum: `nascon_active_ws_connections`, `nascon_active_streams_total`, `nascon_stream_bitrate_kbps`, `nascon_dlq_depth`, and `nascon_qr_scans_total`.
NFR-8	**Guest Web Accessibility**	Guest Web Player complies with WCAG 2.1 AA: contrast ratio $\geq 4.5:1$, full keyboard navigation (Tab/Enter), semantic ARIA labels on media controls, and touch targets $\geq 48 \times 48$ CSS px. Verified by Selenium DOM inspection.
NFR-9	**Privacy & Data Minimization**	QR scan analytics store zero IP addresses, full User-Agent strings, or device fingerprints. Calling DELETE `/audit/v1/privacy/user/{user_id}` wipes PII and anonymizes historical guest/viewer tokens within 1 second.

5. Mandatory Autograder Interface (`/audit/v1/*`)

The Python grading scripts and GitHub Actions runner evaluate your Phase II backend by calling the wrapper endpoints below on `http://localhost:8080`. Just like Day 1, every request and response shape in this section is **binding**: field names, JSON data types, HTTP status codes, error codes, and HMAC signatures are compared literally character-for-character.

5.1 Phase II Endpoint Index

Method & Path	Purpose	Sec.
`GET /health`	Readiness probe (verifies DB, Redis, Media Server, and baseline seed)	5.2
`POST /audit/v1/reset`	Wipes Phase I + Phase II state and re-seeds baseline entities + Day 2 defaults	5.2
`POST /audit/v1/stream/key`	Generate, rotate, or revoke a creator stream key signed with DAY2_SALT	5.4
`POST /audit/v1/costream/session`	Initialize 4-way co-stream session and execute deterministic host-drop handoff	5.5
`GET /audit/v1/qr/{entity_id}`	Resolve dynamic QR metadata, target route, scan count, and export URLs	5.6
`GET /audit/v1/qr/{entity_id}/export.{fmt}`	Download binary/vector QR code in `svg`, `png`, or `pdf` format	5.6
`GET /q/{short_code}`	Public short-link redirect (`302 Found`) to dynamic Guest Web Player route	5.6
`POST /audit/v1/guest/cart-merge`	Verify HTTP 403 guest write block and atomically merge guest cart into member cart	5.7
`POST /audit/v1/moderation/test`	Execute synchronous leetspeak masking, flood kick/mute, and DLQ inspection	5.8
`GET /metrics`	Prometheus text-format observability metrics probe (NFR-6)	5.9
`DELETE /audit/v1/privacy/user/{user_id}`	Right-to-erasure anonymization of user and linked viewer tokens (NFR-9)	5.9

5.2 Global Conventions, Startup & Reset Extensions

- **Startup Command:** The grader runs `DAY1_SALT=<s1> DAY2_SALT=<s2> docker compose up -d --build` from your repo root. All containers (including Selenium Hub, 3 browser nodes, and Media Server) must report healthy and `GET /health` must return `200 {"status": "ok"}` within **120 seconds**.
- **Phase II Baseline Seed (`POST /audit/v1/reset`):** In addition to all Day 1 baseline data (`tenant-alpha`, `tenant-beta`, `prod-jersey-001`, variants `var-jersey-001-M` and `var-jersey-001-L` at `450000` paisa, players `player-alpha-1`, `player-alpha-2`, `player-beta-1`, `player-beta-2`), calling `POST /audit/v1/reset` must also seed:
  - Player `player-beta-2` in `club-beta-1` (if not already present from Day 1).
  - Match `match-m1` (Football, `league-alpha-1`, home `club-alpha-1`, away `club-alpha-2`, initial status `in_progress`, score `1 - 0`).
  - Dynamic QR mapping for `match-m1` (`short_url`: `"http://localhost:8080/q/m1-live"`, initial `scan_count`: `0`), `player-alpha-1` (`short_url`: `"http://localhost:8080/q/p-alpha-1"`), and `club-alpha-1` (`short_url`: `"http://localhost:8080/q/c-alpha-1"`).
  - Anonymous guest token `guest_anon_tok_7721` (valid, unmerged, read-only guest role).
  - Registered user cart for `player-alpha-1` initialized to empty (`items: []`).
- **Validation Order:** Identical to Day 1: (1) malformed JSON or schema violation → `400 VALIDATION_ERROR` (with `details.fields`); (2) referenced entity ID does not exist → `404 *_NOT_FOUND`; (3) permission boundary violated → `403`; (4) business-rule conflict → `409`.
- **Standard Error Envelope:** Every non-2xx response across all Phase II endpoints must return:

```
{
  "error": {
    "code": "GUEST_WRITE_FORBIDDEN",
    "message": "Anonymous guest tokens cannot perform write mutations.",
    "details": { "guest_token": "guest_anon_tok_7721" }
  }
}
```

5.3 Phase II Signature Specification (DAY2_SALT) — Exact Recipe

Every signed Phase II response computes `signature` using HMAC-SHA256 with key `DAY2_SALT` (UTF-8 bytes, never trimmed or hex-decoded) and message `entity_id + "|" + canonical(sig_payload)`. To eliminate any ambiguity about which fields are signed in each endpoint, the exact `entity_id` and `sig_payload` object for each route are specified in the table below:

Endpoint	entity_id	Exact sig_payload Object Passed to canonical()
`POST /audit/v1/stream/key`	`<stream_key>`	`{"action": <req.action>, "player_id": <player_id>, "status": <status>, "stream_key": <stream_key>}`
`POST /audit/v1/costream/session`	`<session_id>`	`{"active_host": <active_host>, "handoff_occurred": <bool>, "layout": <layout>, "participant_count": <int>, "session_id": <session_id>, "status": <status>}`
`GET /audit/v1/qr/{entity_id}`	`<entity_id>`	`{"entity_id": <entity_id>, "short_url": <short_url>, "target_route": <target_route>}`
`POST /audit/v1/guest/cart-merge`	`<guest_token>`	`{"cart_merged": <bool>, "guest_token": <guest_token>, "registered_user_id": <user_id>, "total_items": <int>, "write_rejection_code": <int>}`
`POST /audit/v1/moderation/test`	`<chat_room_id>`	`{"chat_room_id": <chat_room_id>, "flagged_count": <int>, "kicked_users": <kicked_users>, "queue_depth": <int>}`

Canonical JSON Rule: Identical to Day 1 (`json.dumps(sig_payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)`). The output digest is 64 lowercase hexadecimal characters. See Appendix A for worked test vectors.

5.4 Stream Key Lifecycle Management

`POST /audit/v1/stream/key`

Generates, rotates, or revokes a live streaming key for a seeded player.

Request Body:

```
{
  "action": "rotate",
  "player_id": "player-alpha-1",
  "label": "OBS-Main-Desk",
  "stream_key_override": "sk_live_9f83a1b2c3d4e5f6"
}
```

- `action`: Required string. Must be one of `"generate"`, `"rotate"`, or `"revoke"` (else `400 VALIDATION_ERROR`).
- `player_id`: Required string. Must exist in the seeded or active database (else `404 PLAYER_NOT_FOUND`).
- `label`: Optional string (1–64 chars, default `"default"`).
- `stream_key_override`: Optional string. If provided by the autograder during deterministic signature verification, your backend **must** use this exact string as the resulting `stream_key`; otherwise, generate a unique string matching `^sk_live_[a-f0-9]{16}$`.

Response `201 Created` (for `"generate"`) or `200 OK` (for `"rotate"` and `"revoke"`):

```
{
  "player_id": "player-alpha-1",
  "label": "OBS-Main-Desk",
  "stream_key": "sk_live_9f83a1b2c3d4e5f6",
  "status": "active",
  "ingest_endpoints": {
    "rtmp": "rtmp://localhost:1935/live/sk_live_9f83a1b2c3d4e5f6",
    "webrtc": "http://localhost:8889/live/sk_live_9f83a1b2c3d4e5f6/whip"
```

```
  },
  "playback_url": "http://localhost:8888/live/sk_live_9f83a1b2c3d4e5f6/index.m3u8",
  "signature": "e4d909c290d0fb1ca068ffaddf22cbd0..."
}
```

State & Revocation Rules:

- `"generate"`: Creates a new active key (`201`). If the player already has an active key and calls `"generate"` without rotating, replace and invalidate the old key or return the new active key (`201`).
- `"rotate"`: Immediately invalidates any prior key for `player_id`, issues a new `stream_key` (or uses `stream_key_override`), sets `"status": "active"`, and returns `200`.
- `"revoke"`: Revokes the player's current key (if no key exists for that player $\rightarrow$ `404 STREAM_NOT_FOUND`). Returns `200` with `"status": "revoked"`, `"ingest_endpoints": null`, and `"playback_url": null`. Any subsequent attempt to publish media or use a revoked key returns `409 KEY_REVOKED`.

5.5 Synchronized 4-Way Co-Stream Orchestration

`POST /audit/v1/costream/session`

Initializes a multi-broadcaster co-stream session and evaluates host disconnect handoff rules.

Request Body:

```
{
  "session_id": "cs-session-101",
  "host_player_id": "player-alpha-1",
  "participants": ["player-alpha-1", "player-alpha-2", "player-beta-1", "player-beta-2"],
  "layout": "grid_4",
  "simulate_host_drop": true
}
```

Validation & Business Rules (Checked in Exact Order):

1. Schema validation: `session_id` (string), `host_player_id` (string), `participants` (non-empty array of unique strings), `simulate_host_drop` (boolean, default `false`). `layout` (optional string, default `"grid_4"`, allowed values: `"solo"`, `"grid_2"`, `"grid_4"`, `"spotlight"`). Missing/malformed fields $\rightarrow$ `400 VALIDATION_ERROR`.
2. Participant existence: Every player ID in `participants` and `host_player_id` must exist in the database (else `404 PLAYER_NOT_FOUND`). `host_player_id` must be an element of `participants` (else `400 VALIDATION_ERROR`).
3. Maximum capacity check: Co-streaming supports at most 4 concurrent broadcasters. If `len(participants) > 4` $\rightarrow$ `409 COSTREAM_SESSION_FULL`.
4. Layout capacity compatibility: If `layout == "solo"` and effective active participant count > 1, or `layout == "grid_2"` and effective active participant count > 2 $\rightarrow$ `400 UNSUPPORTED_LAYOUT`.
5. **Deterministic Host Drop Handoff:**
   - If `simulate_host_drop == false`: `active_host = host_player_id`, `handoff_occurred = false`, `participant_count = len(participants)`, `active_participants = participants`, `status = "active"`.
   - If `simulate_host_drop == true` and `len(participants) >= 2`: The primary `host_player_id` is removed from the active list. Hosting rights automatically transfer to the first remaining participant in array order (`active_host = remaining[0]`), `handoff_occurred = true`, `participant_count = len(participants) - 1`, `status = "active"`.
   - If `simulate_host_drop == true` and `len(participants) == 1`: The sole host dropped; session terminates cleanly with `active_host = null`, `handoff_occurred = false`, `participant_count = 0`, `status = "terminated"`.

Response `201 Created` (or `200 OK` on idempotent replay of identical request):

```
{
  "session_id": "cs-session-101",
  "status": "active",
  "active_host": "player-alpha-2",
  "handoff_occurred": true,
```

```
  "participant_count": 3,
  "active_participants": ["player-alpha-2", "player-beta-1", "player-beta-2"],
  "layout": "grid_4",
  "composite_playback_url": "http://localhost:8888/costream/cs-session-101/index.m3u8",
  "chat_room_id": "chat-cs-session-101",
  "signature": "c8f2a1b94e7d6c5b4a3f2e1d0c9b8a7f..."
}
```

5.6 Dynamic Cryptographic QR Deep-Linking & Exports

`GET /audit/v1/qr/{entity_id}`

Resolves dynamic QR metadata and increments scan telemetry (unless `?inspect=true` query parameter is passed, which reads metadata without incrementing `scan_count`).

Response `200 OK` (Example for `GET /audit/v1/qr/match-m1`):

```
{
  "entity_id": "match-m1",
  "entity_type": "match",
  "entity_status": "in_progress",
  "short_url": "http://localhost:8080/q/m1-live",
  "target_route": "/web/matches/match-m1/live",
  "deep_link_uri": "nascon://entity/match-m1",
  "scan_count": 1,
  "export_formats": {
    "svg": "http://localhost:8080/audit/v1/qr/match-m1/export.svg",
    "png": "http://localhost:8080/audit/v1/qr/match-m1/export.png",
    "pdf": "http://localhost:8080/audit/v1/qr/match-m1/export.pdf"
  },
  "signature": "d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1..."
}
```

Dynamic Target Mutation Rule (Edge Case 2):

- The `short_url` for an entity remains constant once generated (e.g., `http://localhost:8080/q/m1-live`), so printed physical QR posters never change.
- However, `target_route` (and the `Location` header returned by `GET /q/{short_code}`) must dynamically reflect the entity's current state in the database at the exact millisecond of the scan:
  - Match status `upcoming` → `"/web/matches/{entity_id}/preview"`
  - Match status `in_progress` → `"/web/matches/{entity_id}/live"`
  - Match status `completed` → `"/web/matches/{entity_id}/vod"`
  - Match status `postponed`, `cancelled`, or `abandoned` → `"/web/matches/{entity_id}/notice?state={status}"`
  - Player entity (`player-*`) → `"/web/players/{entity_id}"`
  - Club entity (`club-*`) → `"/web/clubs/{entity_id}"`
  - Co-stream entity (`cs-session-*`) → `"/web/costream/{entity_id}"`
- Notice how `signature` is computed over `entity_id`, `short_url`, and the current `target_route` (Section 5.3). When a match transitions from `in_progress` to `completed` or `postponed`, `target_route` and `signature` update automatically.
- Unknown `entity_id` → `404 QR_NOT_FOUND`.

QR Binary & Vector Export Routes:

- `GET /audit/v1/qr/{entity_id}/export.svg` → Returns `200 OK`, `Content-Type: image/svg+xml`, body starting with `<svg` (or `<?xml`) and encoding the `short_url`.
- `GET /audit/v1/qr/{entity_id}/export.png` → Returns `200 OK`, `Content-Type: image/png`, valid PNG binary starting with the 8-byte PNG magic header.
- `GET /audit/v1/qr/{entity_id}/export.pdf` → Returns `200 OK`, `Content-Type: application/pdf`, valid PDF binary starting with `%PDF-` containing the printable QR poster and `entity_id`.

5.7 Guest Read-Only Boundary & Progressive Cart Merge

`POST /audit/v1/guest/cart-merge`

Verifies that anonymous guest tokens are blocked from write operations (403) and atomically merges staged guest cart items into a registered user's persistent cart upon authentication.

Request Body:

```
{
  "guest_token": "guest_anon_tok_7721",
  "registered_user_id": "player-alpha-1",
  "guest_cart": [
    { "variant_id": "var-jersey-001-L", "quantity": 1 }
  ],
  "test_write_block": true
}
```

Validation & Execution Rules:

1. Validate schema: `guest_token` (non-empty string starting with `guest_`), `registered_user_id` (string), `guest_cart` (array of 1–10 items, each with `variant_id` string and integer `quantity >= 1`), `test_write_block` (boolean, default `true`). Bad schema → `400 VALIDATION_ERROR`.
2. Check entity existence: `registered_user_id` must exist in the seeded users/players table (else `404 PLAYER_NOT_FOUND`). Every `variant_id` in `guest_cart` must exist in the store inventory (else `404 VARIANT_NOT_FOUND`).
3. Check stock availability: If any requested item's merged quantity exceeds current available stock for that variant → `409 OUT_OF_STOCK` (with `details.variant_id`).
4. If `test_write_block == true`, your backend internally simulates a write attempt (e.g., posting a chat message or direct order creation) using `guest_token`, confirms that the RBAC guard rejects it with HTTP `403` (`GUEST_WRITE_FORBIDDEN`), and records `"write_block_verified": true, "write_rejection_code": 403`.
5. Atomically merge `guest_cart` into the persistent cart of `registered_user_id`. Look up each variant's `unit_price_minor` from the seeded catalogue (`450000` paisa for `var-jersey-001-M` and `var-jersey-001-L`). Compute `total_items` as the sum of item quantities in `active_cart.items`.
6. **Network Migration Idempotency (Edge Case 4):** If the exact same (`guest_token`, `registered_user_id`, `guest_cart`) request is replayed (simulating a mobile client switching from 4G to venue Wi-Fi mid-request), return `200 OK` with the already-merged cart state without duplicating item quantities. If the same `guest_token` is reused with a different `registered_user_id` or different payload after already being merged → `409 GUEST_TOKEN_ALREADY_MERGED`.

Response `200 OK`:

```
{
  "guest_token": "guest_anon_tok_7721",
  "write_block_verified": true,
  "write_rejection_code": 403,
  "cart_merged": true,
  "active_cart": {
    "user_id": "player-alpha-1",
    "currency": "PKR",
    "total_items": 1,
    "subtotal_minor": 450000,
    "items": [
      {
        "variant_id": "var-jersey-001-L",
        "quantity": 1,
        "unit_price_minor": 450000
      }
    ]
  },
  "signature": "7f8e9d0c1b2a3f4e5d6c7b8a9f0e1d2c..."
}
```

Direct Write Enforcement Check: In addition to `/audit/v1/guest/cart-merge`, the autograder also

sends direct HTTP requests with header `Authorization: Bearer guest_anon_tok_7721` to `POST /api/v1/chat`, `POST /audit/v1/checkout`, and `POST /audit/v1/events`. All of them must return **HTTP 403** with `error.code == "GUEST_WRITE_FORBIDDEN"`.

5.8 Chat Moderation Engine & Notification DLQ Probe

`POST /audit/v1/moderation/test`

Evaluates synchronous profanity/leetspeak masking, flood detection, idempotent moderator actions, and Dead-Letter Queue (DLQ) depth inspection.

Request Body:

```
{
  "chat_room_id": "chat-cs-session-101",
  "incoming_messages": [
    {
      "message_id": "msg-1",
      "sender": "user-guest-1",
      "text": "Clean sports commentary"
    },
    {
      "message_id": "msg-2",
      "sender": "user-spammer",
      "text": "Watch this illegal stream b@d_w0rd now!"
    },
    {
      "message_id": "msg-3",
      "sender": "user-flooder",
      "text": "GOAL! AAAA!!!"
    }
  ],
  "check_dlq": true,
  "simulate_failed_notifications": 0
}
```

Moderation Rules (Deterministic Behavior Required):

1. **Clean Message:** If a message contains neither profanity/leetspeak tokens nor character flooding, it is appended to `filtered_messages` unchanged with `"flagged": false, "reason": null`.
2. **Leetspeak / Profanity Masking:** Your filter must normalize leetspeak substitutions (`@`→a, `0`→o, `1`/`!`→i, 3→e, `$`→s, `_`/`.`/`-`/`*`→stripped) and match against a dictionary containing at minimum: `badword` (including b@d_w0rd, bad_word), `porn` (p0rn), `scam` (sc@m), `fuck`, `shit`, `bitch`, `asshole`, `idiot_spam`.
   - Every offending token in `text` is replaced in-place with `***` (e.g., `"Watch this illegal stream *** now!"`).
   - The message is included in `filtered_messages` with `"flagged": true, "reason": "profanity_leetspeak"`.
   - The offending `sender` is added to `kicked_users` (deduplicated, preserving order of violation).
3. **Character Repeat / Flood Rejection:** Any message containing ≥ 4 consecutive identical characters (case-insensitive, e.g., AAAA) OR ≥ 3 consecutive exclamation/question marks (`!!!` or `???`) is **dropped completely** from `filtered_messages` (never broadcast to viewers), and its `sender` is added to `muted_users`.
4. **DLQ Inspection:** If `simulate_failed_notifications` (optional int, default `0`) is > 0, enqueue that many failed notification events into the DLQ before reading `dlq_status`. Otherwise (`0`), report current queue_depth (initially `0` after `/reset`) and `undelivered_events`.

Response `200 OK`:

```
{
  "chat_room_id": "chat-cs-session-101",
  "filtered_messages": [
    {
      "message_id": "msg-1",
      "sender": "user-guest-1",
```

```
      "text": "Clean sports commentary",
      "flagged": false,
      "reason": null
    },
    {
      "message_id": "msg-2",
      "sender": "user-spammer",
      "text": "Watch this illegal stream *** now!",
      "flagged": true,
      "reason": "profanity_leetspeak"
    }
  ],
  "kicked_users": ["user-spammer"],
  "muted_users": ["user-flooder"],
  "dlq_status": {
    "queue_depth": 0,
    "undelivered_events": []
  },
  "signature": "3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d..."
}
```

5.9 Observability (`/metrics`) & Privacy Erasure Probes

- **`GET /metrics`:** Returns HTTP `200 OK` with `Content-Type: text/plain; version=0.0.4`. Must contain valid Prometheus gauge/counter lines for `nascon_active_ws_connections`, `nascon_active_streams_total`, `nascon_stream_bitrate_kbps`, `nascon_dlq_depth`, and `nascon_qr_scans_total`.
- **`DELETE /audit/v1/privacy/user/{user_id}`:** Anonymizes personal identifiers for `user_id` and invalidates/unlinks any historical viewer/guest tokens associated with that account. Returns `200 OK`:

```
{
  "user_id": "player-alpha-1",
  "status": "anonymized",
  "pii_retained": false,
  "tokens_wiped": true
}
```

6. Automated Grid QA Harness (Selenium Grid 4, Appium 2, k6 & CI)

Tomorrow's evaluation combines backend contract grading with live automated UI, mobile, and load verification. Nothing in this section may be mocked or stubbed: the autograder queries the live Selenium Grid 4 Hub, executes real headless browser sessions against your Guest Web Player, checks Appium 2 status and deep-link intents, and runs k6.

6.1 Mandatory Standard Port Conventions

All services evaluated by the GitHub Actions runner and desk auditors must bind to the exact host ports listed below. Custom port mappings will cause immediate automated probe failure:

Service Component	Host Endpoint / Port	Verification Probe
Backend Core API & Guest Web	`http://localhost:8080`	`GET /health`, `/audit/v1/*`, `/web/*`, `/metrics`
Selenium Grid 4 Hub	`http://localhost:4444`	`GET http://localhost:4444/status` (`ready: true`, ≥ 3 nodes)
Appium 2 Automation Server	`http://localhost:4723`	`GET http://localhost:4723/status` (`ready: true`)
Media Server RTMP Ingest	`rtmp://localhost:1935`	RTMP publish handshake on `/live/{stream_key}`
Media Server HLS Playback	`http://localhost:8888`	`GET /live/{key}/index.m3u8` & `/costream/{id}/index.m3u8`
Media Server WebRTC (WHIP/WHEP)	`http://localhost:8889`	WebRTC WHIP ingest & WHEP playback endpoints

6.2 Selenium Grid 4 Architecture & Mandatory DOM Selectors

6.2.1 Docker Compose Rules for Selenium Grid 4

To prevent Chromium/Firefox container Out-Of-Memory (OOM) crashes (SIGSEGV / tab crash) on 8 GB RAM runner machines during parallel execution, teams **must** configure `shm_size: "1gb"` (or 2gb) and disable VNC recording under each browser node in `docker-compose.yml`:

```
services:
  selenium-hub:
    image: selenium/hub:4.25.0
    container_name: nascon-selenium-hub
    ports:
      - "4444:4444"
      - "4442:4442"
      - "4443:4443"

  node-chrome-1:
    image: selenium/node-chrome:4.25.0
    shm_size: "1gb"
    depends_on: [selenium-hub]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
      - SE_NODE_MAX_SESSIONS=2
      - SE_START_VNC=false

  node-chrome-2:
    image: selenium/node-chrome:4.25.0
    shm_size: "1gb"
    depends_on: [selenium-hub]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
      - SE_NODE_MAX_SESSIONS=2
      - SE_START_VNC=false

  node-firefox:
    image: selenium/node-firefox:4.25.0
    shm_size: "1gb"
    depends_on: [selenium-hub]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
      - SE_NODE_MAX_SESSIONS=2
```

```
      - SE_START_VNC=false
```

Apple Silicon (M1/M2/M3) Override: On macOS `arm64` laptops where official `selenium/node-*` images may require emulation, teams may use `seleniarm/hub:latest`, `seleniarm/node-chromium:latest`, and `seleniarm/node-firefox:latest`. Regardless of architecture, querying `GET http://localhost:4444/status` must return JSON with `value.ready == true` and `len(value.nodes) >= 3`. Inside the Docker network, browser nodes must be able to reach your backend via `http://host.docker.internal:8080` or your backend service name (`http://backend:8080`).

6.2.2 Mandatory Guest Web Player DOM Contract (`data-testid`)

Because the autograder's Selenium Grid 4 test suite interacts directly with your Guest Web Player in headless Chrome and Firefox, your HTML templates on `/web/matches/{id}/live` (and `/web/costream/{id}`) **must** expose these exact `data-testid` attributes:

data-testid Attribute	HTML Element	Required Behavior Verified by Selenium Grader
`guest-video-player`	`<video>` / `<div>`	Visible media player container rendering the active HLS/WebRTC stream.
`live-score-banner`	`<div>` / `<span>`	Displays real-time match score (e.g., `"1 - 0"`). Must update within 500 ms without page reload when a score event is posted to `/audit/v1/events`.
`pinned-merch-card`	`<div>`	Displays the pinned merchandise item (`prod-jersey-001` / `var-jersey-001-L`).
`add-to-guest-cart-btn`	`<button>`	Clicking adds `var-jersey-001-L` (qty 1) to the guest's staging cart and increments `guest-cart-count`.
`guest-cart-count`	`<span>`	Displays integer count of items currently in the guest cart (e.g., `"0"` $\rightarrow$ `"1"`).
`chat-message-list`	`<div>` / `<ul>`	Scrollable container rendering live chat messages. Masked profanity appears as `***`.
`chat-input-field`	`<input>`	Text input for chat messages.
`chat-send-btn`	`<button>`	When clicked by an unauthenticated guest, triggers a write attempt that is blocked (`403`) and opens the progressive registration modal (`progressive-auth-modal`).
`progressive-auth-modal`	`<div>`	Modal dialog prompting the guest to log in or register to complete a write action or checkout, preserving their staged cart.

6.2.3 Required Selenium Test Cases (`tests/selenium/test_guest_grid.py`)

Your repository must include a runnable pytest/Selenium suite executing in parallel across at least 3 Grid sessions (`pytest -n 3 tests/selenium/`) verifying:

1. **Test 1 (QR Resolution & Zero-Friction Load):** Navigate to `http://localhost:8080/q/m1-live`, verify HTTP redirect to `/web/matches/match-m1/live`, assert `[data-testid="guest-video-player"]` and `[data-testid="live-score-banner"]` render within 3 seconds with zero login modal visible on initial load.
2. **Test 2 (Guest Write Block & Progressive Modal):** In a fresh guest session, type a message into `[data-testid="chat-input-field"]` and click `[data-testid="chat-send-btn"]`; assert that the message is not appended to `chat-message-list` and `[data-testid="progressive-auth-modal"]` becomes visible.
3. **Test 3 (Guest Cart Staging & Real-Time Score Sync):** Click `[data-testid="add-to-guest-cart-btn"]`, verify `[data-testid="guest-cart-count"]` updates to `"1"`, trigger a score event via POST `/audit/v1/events`, and assert `[data-testid="live-score-banner"]` updates in the browser DOM within 500 ms.

6.3 Appium 2 Mobile Automation & Deep-Link Standards

- **Server & Driver Setup:** Appium 2 must run on `http://localhost:4723` with the `uiautomator2` driver (Android) or `xcuitest` driver (iOS) installed. Calling `GET http://localhost:4723/status` must return `200 OK` with `"ready": true`.
- **Android Manifest Intent-Filter Requirement:** Your Android app (`AndroidManifest.xml`) must register a browsable intent-filter for scheme `nascon`:

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="nascon" />
</intent-filter>
```

- **Deep-Link Verification Command:** During the desk audit and Appium test execution (`tests/appium/test_mobile_deeplink.py`), the runner fires:

```
adb shell am start -W -a android.intent.action.VIEW -d "nascon://entity/match-m1"
adb shell am start -W -a android.intent.action.VIEW -d "nascon://stream/cs-session-101"
```

  The native app must come to the foreground within 2 seconds and open the corresponding Match Live / Co-Stream screen directly without crashing or stopping on the login screen.
- **Cross-Surface Integration Test:** Your test suite must execute at least one cross-surface scenario where the mobile app initiates or joins stream `cs-session-101` via Appium while a simultaneous Selenium Grid browser session opens `/web/costream/cs-session-101` and verifies synchronized chat delivery within 3.0 seconds.

6.4 k6 High-Concurrency Load Specification (NFR-2)

Teams must commit the exact load profile below in `tests/k6_load.js`. The autograder executes `k6 run --summary-export=tests/reports/k6-summary.json tests/k6_load.js`:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    guest_viewers: {
      executor: 'constant-vus',
      vus: 500,
      duration: '15s',
      exec: 'viewStream',
    },
    chat_posters: {
      executor: 'constant-vus',
      vus: 50,
      duration: '15s',
      exec: 'postChat',
    },
  },
  thresholds: {
    http_req_failed: ['rate==0.00'],
    http_req_duration: ['p(95)<800'],
  },
};

export function viewStream() {
  const res = http.get('http://localhost:8080/audit/v1/qr/match-m1?inspect=true');
  check(res, {
    'guest QR status is 200': (r) => r.status === 200,
    'has target_route': (r) => r.json('target_route') !== undefined,
  });
  sleep(0.1);
}

export function postChat() {
```

```
  const payload = JSON.stringify({
    chat_room_id: 'chat-cs-session-101',
    incoming_messages: [
      { message_id: `k6-${__VU}-${__ITER}`, sender: `user-${__VU}`, text: 'Live reaction' }
    ],
    check_dlq: false,
  });
  const res = http.post('http://localhost:8080/audit/v1/moderation/test', payload, {
    headers: { 'Content-Type': 'application/json' },
  });
  check(res, {
    'chat status is 200 or 201': (r) => r.status === 200 || r.status === 201,
  });
  sleep(0.2);
}
```

Note on Rate-Limiting During k6: Just like Day 1, `/audit/v1/*` routes are exempt from the 100 req/10s IP rate limiter so the 550 concurrent virtual users in k6 can stress-test your concurrency pool without triggering false HTTP 429 failures.

6.5 Multi-Laptop Workload Distribution Architecture (Labs C-401 – C-404)

To prevent CPU thermal throttling or RAM exhaustion on a single machine during Desk Audit & Live Grading in venues **C-401 – C-404**, teams may either run the full stack on one high-spec machine OR distribute workloads across up to 3 team laptops connected to the same local LAN / team Wi-Fi hotspot:

- **Laptop 1 (Primary Node — Backend Platform & Media Server):** Runs Docker Compose with PostgreSQL, Redis, MinIO, MediaMTX/LiveKit, and Core API (`8080`, 1935, 8888, 8889).
- **Laptop 2 (Mobile QA Node — Appium 2 & Physical Phone):** Physical Android/iOS device connected via USB debugging running Appium 2 on port 4723 (forwarded or pointed at Laptop 1's LAN IP).
- **Laptop 3 (Browser Grid & Load Node — Selenium Grid 4 & k6):** Runs Selenium Hub + 3 browser nodes (4444) and k6 targeting Laptop 1's LAN IP (or via SSH local port forwarding `ssh -L 8080:localhost:8080` so all `localhost` URLs resolve transparently).

Important for Cloud GitHub Actions Autograder: While you may distribute across 3 laptops during the physical desk check, your committed `docker-compose.yml` in the repository root **must** define the complete stack (including `selenium-hub` and 3 browser nodes on port 4444) so the automated GitHub Actions runner can bring up everything on a single clean Ubuntu runner with `docker compose up -d --build`.

7. Phase II Evaluation Weightage (80% Backend / 20% Frontend)

7.1 Mandatory 80% Backend / 20% Frontend Marks Distribution

Across Phase II evaluation, total marks are strictly distributed as **80% Backend Platform & Automated QA Infrastructure** and **20% Frontend (Mobile Application UI + Guest Web Player UI)**:

Evaluation Category	Weightage	Graded Surfaces & Verification Scope
Backend Platform, Contracts, Load & Grid Architecture	**80%**	All `/audit/v1/*` REST contracts, DAY2_SALT HMAC signatures, RTMP/WebRTC/HLS media pipeline, 4-way co-stream handoff state machine, dynamic QR server routing & SVG/PNG/PDF generation, HTTP 403 guest write-block guard, atomic cart merge, leetspeak/flood moderation & DLQ, Prometheus `/metrics`, NFR-1 WebSocket latency ($p_{95} < 500$ ms), NFR-2 k6 550-VU load test, Selenium Grid 4 multi-node backend orchestration, and Backend Architecture Defense in Viva.
Frontend Surfaces (Native Mobile UI + Guest Web UI)	**20%**	Guest Web Player responsive UI (`/web/*`), video player overlay rendering, WCAG 2.1 AA accessibility, progressive login/registration modal UX, Native Mobile App broadcaster/co-stream UI, QR scanner & Android Intent-Filter (`nascon://`) foreground navigation via Appium 2, and Frontend/HCI Viva verification.
Total Phase II Evaluation	**100%**	**Automated Autograder + Desk QA Verification + Top-8 Finalist Viva**

7.2 Automated Vector Evaluation & Coverage File (COVERAGE_DAY2.json)

Just like Day 1, each backend module is evaluated against hidden test vectors where a vector passes only if every assertion holds (status code, error code, required JSON keys, and HMAC signature), combined with unit-test line coverage ($\geq 85\%$). Commit COVERAGE_DAY2.json at the repository root with line-coverage percentages for the five Phase II modules:

```
{
  "streaming": 89.5,
  "costream": 87.0,
  "qr_guest": 91.2,
  "moderation_dlq": 88.4,
  "observability": 86.0
}
```

7.3 Disqualification & Compliance Rules

- **Reproducible Docker & Grid Stack:** `docker compose up -d --build` must start cleanly within 120 seconds, and `http://localhost:4444/status` must report at least 3 ready browser nodes.
- **Phase I Commit Ancestry:** Your Day 2 branch must descend directly from Saturday's locked DAY1_BASE_SHA.
- **Server-Side Security:** Hiding buttons in the frontend UI without enforcing HTTP 403 GUEST_WRITE_FORBIDDEN on the backend API, or hard-coding DAY2_SALT in source files instead of reading from environment variables, voids credit for the affected module.

8. Phase II Hidden Complexity Scenarios

These four edge cases are tested automatically by the hidden grading vectors and probed live during the Top-8 Finalist Viva at 14:30 PKT:

#	Scenario	Required Deterministic System Behavior
1	**Host Abrupt Disconnection Mid-Broadcast**	When the primary host of a 4-way co-stream drops (`simulate_host_drop: true` or WebSocket disconnect), `/audit/v1/costream/session` promotes `participants[1]` to `active_host`, sets `handoff_occurred: true`, decrements `participant_count` to 3, keeps `composite_playback_url` alive without resetting `chat_room_id`, and leaves zero zombie FFmpeg/WebRTC processes.
2	**Dynamic QR Target Mutation Post-Print**	A physical QR poster (`/q/m1-live`) is printed while `match-m1` is `in_progress` (resolving to `/web/matches/match-m1/live`). The grader then transitions `match-m1` to `completed` (via POST `/audit/v1/events match_end`) or `postponed`. Immediately re-calling GET `/audit/v1/qr/match-m1` must return the identical `short_url` but an updated `target_route` (`/web/matches/match-m1/vod` or `/notice?state=postponed`) and a freshly computed `signature`.
3	**Dual-Moderator Concurrent Kick/Mute Race**	The autograder fires 10 simultaneous threads to POST `/audit/v1/moderation/test` (and concurrent moderator mute actions) targeting the exact same offending user `user-spammer` at the same millisecond. Your backend must resolve idempotently without database deadlocks (`500`), duplicate entries in `kicked_users` (must be a deduplicated list `["user-spammer"]`), or corrupted DLQ counters.
4	**Network Migration During Progressive Cart Merge**	A guest user roaming between mobile data (4G/5G) and venue Wi-Fi in **C-401 – C-404** submits POST `/audit/v1/guest/cart-merge` twice with the same `guest_token` and payload due to a dropped TCP ACK. The second call must return `200 OK` with `total_items: 1` (never 2) and must not double-reserve inventory.

A. Reference Code and Cryptographic Test Vectors

Use this exact Python reference function to verify your DAY2_SALT HMAC-SHA256 signatures before pushing:

```
import hmac, hashlib, json

def canonical(obj) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def sign_day2(day2_salt: str, entity_id: str, sig_payload: dict) -> str:
    msg = (entity_id + "|" + canonical(sig_payload)).encode("utf-8")
    return hmac.new(day2_salt.encode("utf-8"), msg, hashlib.sha256).hexdigest()
```

Worked Test Vectors (Using Reference Salt `T01D27ad7ffff12e`)

1. **Vector 1 — Stream Key (`POST /audit/v1/stream/key`):**
   - DAY2_SALT: `"T01D27ad7ffff12e"`
   - `entity_id`: `"sk_live_9f83a1b2c3d4e5f6"`
   - `sig_payload`:
     ```
     {
       "action": "rotate",
       "player_id": "player-alpha-1",
       "status": "active",
       "stream_key": "sk_live_9f83a1b2c3d4e5f6"
     }
     ```
   - Message string passed to HMAC (single line with no spaces):

`sk_live_9f83a1b2c3d4e5f6|{"action":"rotate","player_id":"player-alpha-1","status":"active","stream_key":"sk_live_9f83a1b2c3d4e5f6"}`

- Expected `signature`: deterministic 64-character lowercase hex output of `sign_day2` above.

2. **Vector 2 — Legacy 2-Key Stream Vector (Backward-Compatible Support):**
   - `DAY2_SALT`: `"T01D27ad7ffff12e"`, `entity_id`: `"sk_live_101"`
   - `sig_payload`: `{"action": "rotate", "player_id": "player-alpha-1"}`
   - Message string: `sk_live_101|{"action":"rotate","player_id":"player-alpha-1"}`
   - Note: To ensure zero friction if a team implemented the short payload from the preview specification, the autograder accepts **either** the full 4-key `sig_payload` (Table 5.3) **or** the 2-key `{"action", "player_id"}` payload on `/audit/v1/stream/key`.

B. Error Code Catalogue (Part II)

All Phase I error codes from Day 1 Appendix B remain active. Phase II adds the following machine-readable `error.code` strings:

HTTP	Code String	Trigger Condition
400	`VALIDATION_ERROR`	Malformed JSON, missing required field, invalid `action`, or `host_player_id` not in `participants` (`details.fields` lists offending fields).
400	`UNSUPPORTED_LAYOUT`	Requested co-stream `layout` (`solo`, `grid_2`) cannot fit active participant count.
401	`UNAUTHENTICATED`	Missing, malformed, expired, or tampered JWT token on protected routes.
403	`GUEST_WRITE_FORBIDDEN`	Anonymous guest token (`guest_anon_tok_*`) used to attempt any write operation (chat post, checkout, follow, score/roster mutation).
403	`FORBIDDEN`	Authenticated member attempting an action outside their 5-tier RBAC scope.
404	`PLAYER_NOT_FOUND`	Referenced `player_id` or `registered_user_id` does not exist.
404	`STREAM_NOT_FOUND`	Target `stream_key` or `session_id` does not exist (e.g., revoking a non-existent key).
404	`QR_NOT_FOUND`	Requested `entity_id` or `short_code` has no registered dynamic QR mapping.
404	`VARIANT_NOT_FOUND`	A `variant_id` in `guest_cart` does not exist in store inventory.
409	`COSTREAM_SESSION_FULL`	Attempting to initialize or join a co-stream with > 4 participants.
409	`KEY_REVOKED`	Attempting stream ingest or playback activation using a revoked `stream_key`.
409	`GUEST_TOKEN_ALREADY_MERGED`	Re-using an already merged `guest_token` with a different user or different cart payload.
409	`OUT_OF_STOCK`	Merging or checking out a cart item whose quantity exceeds available variant stock.
413	`PAYLOAD_TOO_LARGE`	Request body exceeds 1 MB.
429	`RATE_LIMITED`	Public `/api/v1/*` rate limit (100 req / 10s) or chat flood threshold exceeded.

C. Pre-Submission Final Checklist (Labs C-401 – C-404)

Before **14:00:00 PKT Sharp**, verify every item below at your venue desk (**C-401 – C-404**):

- [ ] `DAY1_SALT=<s1> DAY2_SALT=<s2> docker compose up -d --build` starts cleanly and `GET /health` returns `200` within 120 seconds.
- [ ] `git merge-base --is-ancestor $DAY1_BASE_SHA HEAD` passes (Saturday commit history is intact).
- [ ] `POST /audit/v1/reset` seeds `match-m1`, `player-beta-2`, `guest_anon_tok_7721`, and default QR mappings.
- [ ] Every `/audit/v1/*` endpoint returns the exact JSON keys, HTTP status codes, and `DAY2_SALT` HMAC signatures defined in Section 5.
- [ ] `GET http://localhost:4444/status` returns `ready: true` with at least **3 active browser nodes** (`shm_size: "1gb"`).
- [ ] Guest Web Player on `/web/matches/match-m1/live` includes all 9 mandatory `data-testid` attributes (Section 6.2.2).
- [ ] `GET http://localhost:4723/status` returns `ready: true` and `adb shell am start -W -a android.intent.action.VIEW -d "nascon://entity/match-m1"` opens the native app screen.
- [ ] `k6 run tests/k6_load.js` completes 15 seconds with 500 guest VUs + 50 chat VUs at **0% HTTP 5xx errors**.
- [ ] Test reports (`tests/reports/*`), `COVERAGE_DAY2.json`, and `TRACEABILITY.md` are committed and pushed before **14:05:00 PKT Sharp**.

If your code works on the first try, check again. It's probably the wrong problem.

Understanding The Problem Is Also Part Of The Problem.